HANZO GPU INFERENCE

GPU-Accelerated AI Inference on Hanzo Cloud

Zach Kelling

Hanzo AI

March 2026

Abstract

We present the GPU inference architecture of Hanzo Cloud, a production system serving over 47 million inference requests per month across heterogeneous GPU clusters. The system addresses four challenges in large-scale model serving: (i) batch inference with adaptive batching that maximizes GPU utilization while meeting latency SLAs, (ii) multi-model GPU sharing via spatial and temporal multiplexing on a single device, (iii) GPU-aware scheduling that accounts for memory bandwidth, compute capacity, and thermal constraints, and (iv) latency optimization through speculative decoding, KV-cache management, and continuous batching. We describe the architecture deployed on Hanzo ACI nodes, where GPU operators stake HANZO tokens and serve Zen foundation models (Qwen3+). Production measurements on NVIDIA H100 and A100 clusters demonstrate 3.2× throughput improvement over naive serving, p99 latency under 180ms for 7B-parameter models, and 94% average GPU utilization across the fleet. The system integrates with Hanzo Gateway for authentication and routing, Hanzo IAM for tenant isolation, and the Candle runtime for Rust-native inference kernels.

1 Introduction

GPU inference at scale presents a fundamental tension: maximizing throughput (requests per second per GPU) requires batching, but batching increases latency for individual requests. Naive approaches—static batch sizes, one-model-per-GPU allocation, FIFO scheduling—waste GPU cycles and fail to meet the diverse latency requirements of production workloads [9, 8].

Hanzo Cloud serves a heterogeneous workload: real-time chat completions (p99 $<$ 200ms TTFT), batch document processing (throughput-optimized), agent tool calls (latency-sensitive), and embedding generation (compute-light). Each workload has different optimal serving parameters. A single static configuration cannot serve all efficiently.

This paper describes three core contributions:

1. **Adaptive continuous batching** (§3): A scheduler that dynamically adjusts batch composition based on request priority, sequence length distribution, and GPU memory pressure.

2. **Multi-model spatial sharing** (§4): A memory manager that co-locates multiple models on a single GPU, switching between them with sub-millisecond overhead via pre-allocated memory partitions.

3. **GPU-aware cluster scheduling** (§5): A cluster-level scheduler that routes requests to optimal GPU nodes based on model affinity, current load, thermal state, and network topology.

The system is deployed on Hanzo ACI [1] infrastructure, where GPU node operators participate in a staked compute market. Integration with the Candle ML framework [2] provides Rust-native inference kernels, and the Jin architecture [3] handles multimodal routing.

2 System Architecture

The inference stack consists of four layers, each with well-defined interfaces.

Definition 1 (Inference Request)**.** *An inference request $r = (m, \mathbf{x}, \theta, \tau)$ consists of a model identifier m, input token sequence $\mathbf{x} \in \mathcal{V}^n$, generation parameters θ (temperature, top-p, max tokens), and a latency class $\tau \in \{\texttt{realtime}, \texttt{standard}, \texttt{batch}\}$.*

2.1 Request Ingress

Requests enter through Hanzo Gateway [4], which performs:

1. **Authentication**: JWT validation via Hanzo IAM. Organization extracted from the `owner` claim. All subsequent operations scoped to the authenticated org.
2. **Rate limiting**: Token-bucket rate limiting per org, with burst allowance proportional to subscription tier.
3. **Model resolution**: Maps model aliases (e.g., `zen-large`) to specific model versions and quantization levels.
4. **Priority assignment**: Determines latency class τ from request headers and org SLA configuration.

2.2 Router Layer

The router maintains a real-time view of cluster state and selects the optimal GPU node for each request.

Definition 2 (GPU Node State)**.** *A GPU node g exposes state $\sigma_g = (M_g, U_g, Q_g, T_g)$ where M_g is the set of loaded models, $U_g \in [0, 1]$ is current utilization, Q_g is queue depth, and T_g is GPU temperature in Celsius.*

The routing function minimizes expected latency subject to load balancing:

$$g^* = \arg\min_{g \in \mathcal{G}} \left[\alpha \cdot \hat{L}(r, g) + \beta \cdot U_g + \gamma \cdot \mathbb{1}[m \notin M_g] \cdot C_{\text{load}} \right], \tag{1}$$

where $\hat{L}(r, g)$ is estimated latency, C_{load} is the cold-load penalty, and α, β, γ are tunable weights.

2.3 Inference Engine

Each GPU node runs the Candle-based inference engine with the following components:

- **Model registry**: In-GPU-memory model cache with LRU eviction.
- **KV-cache manager**: PagedAttention [8] implementation managing key-value cache blocks across concurrent sequences.
- **Batch scheduler**: Continuous batching engine that adds and removes sequences from the running batch at each iteration step.
- **Decode kernel**: Fused attention + sampling kernel compiled for the specific GPU architecture (SM90 for H100, SM80 for A100).

2.4 Metrics and Observability

Every inference request emits structured metrics to Prometheus:

- Time-to-first-token (TTFT), inter-token latency (ITL), total latency.
- GPU memory utilization, compute utilization, power draw.
- Batch size, queue depth, cache hit rate.
- Per-org request count, token count, error rate.

3 Adaptive Continuous Batching

Static batching wastes GPU cycles: short sequences pad to the longest sequence in the batch, and completed sequences block new admissions until the entire batch finishes. Continuous batching [9] solves this by admitting and evicting sequences at each decode step.

3.1 Iteration-Level Scheduling

At each decode iteration t, the scheduler decides which sequences to include in the batch $\mathcal{B}_t$:

$$\mathcal{B}_t = \arg\max_{\mathcal{B} \subseteq \mathcal{R}_t} \sum_{r \in \mathcal{B}} w(r) \quad \text{s.t.} \quad \sum_{r \in \mathcal{B}} k(r) \leq K_{\max}, \tag{2}$$

where $\mathcal{R}_t$ is the set of runnable sequences, $w(r)$ is priority weight, and $k(r)$ is the KV-cache block count for sequence r. $K_{\max}$ is the total available cache blocks.

Proposition 1 (Throughput Bound)**.** *Under continuous batching with $K_{\max}$ cache blocks and average sequence length $\bar{n}$, the maximum sustainable throughput is:*

$$\Theta_{\max} = \frac{K_{\max}}{\bar{n}} \cdot f_{iter}, \tag{3}$$

where f_{iter} is the iteration frequency (decode steps per second).

3.2 Priority-Aware Batching

Not all requests are equal. Realtime requests (chat) must preempt batch requests (document processing). We assign priority weights:

$$w(r) = \begin{cases} 10.0 & \text{if } \tau_r = \texttt{realtime}, \\ 1.0 & \text{if } \tau_r = \texttt{standard}, \\ 0.1 & \text{if } \tau_r = \texttt{batch}. \end{cases} \tag{4}$$

When GPU memory is contiguous, realtime requests preempt batch sequences via KV-cache swapping to CPU memory. The preempted sequence resumes when GPU memory becomes available.

3.3 Prefill–Decode Disaggregation

Prefill (processing input tokens) and decode (generating output tokens) have fundamentally different compute profiles. Prefill is compute-bound; decode is memory-bandwidth-bound. We separate them:

1. **Prefill phase**: Dedicated GPU SMs process the input sequence in a single forward pass, populating the KV-cache.
2. **Decode phase**: The sequence joins the continuous batch for autoregressive generation.

On H100 GPUs with 80GB HBM3, this disaggregation improves prefill throughput by 2.1× and reduces TTFT by 45% compared to unified processing.

4 Multi-Model GPU Sharing

Dedicating one GPU per model wastes capacity. A 7B-parameter model in FP16 occupies 14GB; an 80GB H100 can host multiple models simultaneously.

4.1 Spatial Multiplexing

We partition GPU memory into fixed regions:

- **Model weights region**: Pinned allocations for model parameters. Multiple models co-reside if total weight size fits.
- **KV-cache pool**: Shared pool of cache blocks, allocated dynamically across models based on demand.
- **Workspace**: Scratch memory for intermediate activations, shared across all models (only one model computes at a time per SM group).

Definition 3 (Memory Budget)**.** *For a GPU with total memory M_{total}, the allocation is:*

$$M_{total} = \sum_{i=1}^{N} M_{weights}^{(i)} + M_{kv} + M_{workspace} + M_{reserved}, \tag{5}$$

where N is the number of co-located models, M_{kv} is the shared KV-cache pool, $M_{workspace}$ is the activation workspace, and $M_{reserved}$ is a safety margin (typically 2GB).

4.2 Temporal Multiplexing

When models cannot co-reside (e.g., two 70B models on one GPU), we use temporal multiplexing with fast model swapping:

1. Model weights are kept in host pinned memory (PCIe-accessible).
2. On model switch, weights are streamed via PCIe Gen5 at 64 GB/s.
3. A 14GB model swap completes in ∼220ms. KV-cache is preserved across swaps for the active model's in-flight sequences.

Theorem 2 (Sharing Efficiency)**.** *Let λ_i be the request rate for model i and μ_i be its service rate on a dedicated GPU. Under spatial multiplexing with N models, the effective service rate for model i is:*

$$\mu_i^{shared} = \mu_i \cdot \frac{M_{kv}^{(i)}}{M_{kv}}, \tag{6}$$

where $M_{kv}^{(i)}$ is the KV-cache allocation for model i. Sharing is efficient when $\sum_i \lambda_i/\mu_i^{shared} < 1$.

5 GPU-Aware Cluster Scheduling

The cluster scheduler operates at the request level, assigning incoming requests to GPU nodes. Unlike traditional load balancers, it must account for GPU-specific constraints.

5.1 Node Health Scoring

Each GPU node reports a health score h_g computed as:

$$h_g = \omega_1(1 - U_g) + \omega_2 \frac{M^g_{\text{free}}}{M^g_{\text{total}}} + \omega_3 \left(1 - \frac{T_g}{T_{\max}}\right) + \omega_4 \frac{1}{1 + Q_g}, \tag{7}$$

where $\omega_1 + \omega_2 + \omega_3 + \omega_4 = 1$ are configurable weights. In production, we use $\omega = (0.3, 0.3, 0.1, 0.3)$.

5.2 Model Affinity

Loading a model onto a GPU that does not already have it incurs a cold-start penalty. The scheduler maintains an affinity matrix $A \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{G}|}$ where $A_{m,g} = 1$ if model m is loaded on GPU g. The routing cost (Equation 1) penalizes routing to nodes without model affinity.

5.3 Thermal Throttling Avoidance

GPU thermal throttling reduces clock speed by up to 30% when junction temperature exceeds 83°C (H100). The scheduler proactively avoids hot nodes:

$$\text{throttle}(g) = \begin{cases} 1.0 & \text{if } T_g < 75°\text{C}, \\ 1 - 0.3 \cdot \frac{T_g - 75}{T_{\max} - 75} & \text{if } T_g \geq 75°\text{C}. \end{cases} \tag{8}$$

5.4 Multi-GPU Tensor Parallelism

For models exceeding single-GPU memory (70B+ parameters), the scheduler co-locates tensor-parallel shards on GPUs connected via NVLink:

- **Intra-node**: NVLink at 900 GB/s (H100 SXM). Shards on the same physical node.
- **Inter-node**: InfiniBand at 400 Gb/s. Used only when intra-node GPUs are exhausted.

The scheduler ensures that all shards for a tensor-parallel group are placed on GPUs with symmetric interconnect bandwidth to avoid stragglers.

6 Latency Optimization

6.1 Speculative Decoding

Speculative decoding uses a small draft model to propose k tokens, which the target model verifies in a single forward pass. If all k tokens are accepted, we achieve $k\times$ speedup for those tokens.

Definition 4 (Acceptance Rate)**.** *Let $p(x_t|x_{<t})$ be the target model distribution and $q(x_t|x_{<t})$ be the draft model distribution. The acceptance probability for token x_t is:*

$$\alpha(x_t) = \min\left(1, \frac{p(x_t|x_{<t})}{q(x_t|x_{<t})}\right). \tag{9}$$

With a well-aligned draft model (Qwen3-0.6B drafting for Qwen3-32B), we achieve an average acceptance rate of 0.78 with $k = 5$ speculative tokens, yielding 2.4× decode speedup.

6.2 KV-Cache Compression

Long sequences consume proportionally large KV-cache. We apply two compression techniques:

1. **Grouped-query attention (GQA)**: Models use 8 KV heads instead of 32, reducing cache size by 4×.
2. **Quantized cache**: KV-cache entries are stored in FP8 (E4M3) instead of FP16, halving cache memory with negligible quality loss (<0.1% perplexity increase on validation sets).

Combined, these reduce per-sequence cache from 2.1MB to 262KB for a 7B model at 4096 context length, enabling 8× more concurrent sequences.

6.3 Prefix Caching

Many requests share common prefixes (system prompts, few-shot examples). We hash prefix token sequences and cache their KV states:

$$\text{cache_key} = \texttt{SHA256}(m \| \mathbf{x}_{1:j}), \tag{10}$$

where j is the prefix boundary. Cache hits skip prefill for the shared prefix, reducing TTFT by 60–80% for repeated system prompts.

7 Production Results

We report results from 90 days of production operation (January–March 2026) across a fleet of 128 H100 GPUs and 64 A100 GPUs serving Zen models.

Metric	Before	After	Improvement
Throughput (req/s/GPU)	42	134	3.2×
p50 TTFT (ms)	85	31	2.7×
p99 TTFT (ms)	520	178	2.9×
GPU utilization (%)	61	94	+33pp
Models per GPU	1.0	3.2	3.2×
Cost per 1M tokens ($)	0.42	0.13	3.2×

Table 1: Production metrics before and after the optimizations described in this paper. "Before" is the baseline Candle serving with static batching.

7.1 Batch Size Distribution

Adaptive batching produces a bimodal batch size distribution: realtime requests form small batches (4–16 sequences), while batch workloads fill to the maximum (64–128 sequences). The scheduler correctly prioritizes realtime requests, achieving p99 TTFT $<$ 200ms even at 90% GPU utilization.

7.2 Multi-Model Sharing Efficiency

On a typical H100 node, we co-locate Qwen3-7B (14GB), Qwen3-1.5B (3GB), and an embedding model (1.2GB), leaving 61.8GB for KV-cache and workspace. This configuration serves 95% of production traffic without model swapping.

7.3 Speculative Decoding Impact

Speculative decoding with Qwen3-0.6B as draft model reduces median inter-token latency from 18ms to 7.5ms for Qwen3-32B, a 2.4× improvement. Acceptance rates vary by task: code generation achieves 0.85 (highly predictable), creative writing achieves 0.62 (less predictable).

8 Integration with Hanzo Infrastructure

The GPU inference system integrates with the broader Hanzo platform:

- **Hanzo ACI** [1]: GPU operators register as ACI compute nodes, staking HANZO tokens proportional to declared GPU capacity.
- **Hanzo Gateway** [4]: All inference requests enter through the gateway, which handles authentication, rate limiting, and org-scoped routing.
- **Hanzo IAM** [5]: Tenant isolation ensures that KV-cache, model state, and metrics are never shared across organizations.
- **Hanzo Edge Inference** [6]: For latency-critical deployments, a subset of models are served at edge locations with the same scheduling framework.
- **Candle Runtime** [2]: Rust-native inference kernels compiled per GPU architecture, eliminating Python overhead.

9 Related Work

vLLM [8] introduced PagedAttention for efficient KV-cache management. We adopt this technique and extend it with FP8 quantized cache and cross-model cache pool sharing.

Orca [9] proposed iteration-level scheduling for continuous batching. Our system adds priority-aware preemption and prefill–decode disaggregation.

TensorRT-LLM [10] provides optimized inference kernels but requires NVIDIA-specific compilation. Our Candle-based approach achieves comparable kernel performance with a more portable Rust codebase.

Lux Network [7] provides the underlying consensus layer for ACI compute attestation, ensuring that inference results can be verified on-chain.

10 Conclusion

We have described the GPU inference architecture of Hanzo Cloud, achieving 3.2× throughput improvement and p99 latency under 180ms through adaptive continuous batching, multi-model GPU sharing, and GPU-aware cluster scheduling. The system serves 47M+ monthly inference requests across 192 GPUs with 94% average utilization. All optimizations are deployed in production and integrated with the Hanzo ACI compute market, where GPU operators earn HANZO tokens for verified inference work.

References

[1] Hanzo AI Research. Hanzo ACI: AI Chain Infrastructure for decentralized compute markets. *Hanzo AI Research*, December 2024.

[2] Hanzo AI Research. Hanzo Candle: Rust-native ML framework for production inference. *Hanzo AI Research*, 2024.

[3] Hanzo AI Research. Jin: Unified multimodal AI architecture. *Hanzo AI Research*, 2025.

[4] Hanzo AI Research. Hanzo Gateway: API gateway with KrakenD backend. *Hanzo AI Research*, 2024.

[5] Hanzo AI Research. Hanzo IAM: Identity and access management for AI platforms. *Hanzo AI Research*, 2024.

[6] Hanzo AI Research. Hanzo Edge Inference: On-device and edge AI serving. *Hanzo AI Research*, 2025.

[7] Lux Partners. Lux: A scalable multi-consensus blockchain with post-quantum cryptography. *Lux Network Technical Report*, 2024.

[8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proc. SOSP*, 2023.

[9] G. Yu, J. Jeong, G. Kim, S. Kim, and B. Chun. Orca: A distributed serving system for Transformer-based generative models. In *Proc. OSDI*, 2022.

[10] NVIDIA. TensorRT-LLM: High-performance inference for large language models. *NVIDIA Developer Documentation*, 2024.